

Regular Expressions (Regex)

Regex is used to search, match, validate, split, or replace text according to a pattern.

Java provides regex mainly through: `import java.util.regex.*;`

There are two important classes:

Matcher → performs matching on the text

Pattern → represents the regex pattern

Regex Symbol / Format	Meaning	Example	Matches
.	Any single character	a.c	abc, axc
^	Beginning of string	^Java	Java is easy
\$	End of string	Java\$	I like Java
*	0 or more occurrences	ab*	a, ab, abb
+	1 or more occurrences	ab+	ab, abb
?	0 or 1 occurrence	ab?	a, ab
{n}	Exactly n occurrences	a{3}	aaa
{n,}	At least n occurrences	a{2,}	aa, aaa
{n,m}	n to m occurrences	a{2,4}	aa, aaa, aaaa
[abc]	Any one of a, b, c	[abc]	a, b, c
[^abc]	Anything except a, b, c	[^abc]	d, x, 1
[a-z]	Lowercase letter	[a-z]	a to z
[A-Z]	Uppercase letter	[A-Z]	A to Z
[0-9]	Digit	[0-9]	0 to 9
[a-zA-Z]	Any alphabet	[a-zA-Z]	A, a
\d	Digit	\d	0 to 9
\D	Non-digit	\D	A, @
\s	Whitespace	\s	space, tab
\S	Non-whitespace	\S	A, 5
\w	Word character	\w	letters, digits, _
\W	Non-word character	\W	@, #, !
()	Capturing group	(abc)	abc
\.	Literal dot	\.	.
\\	Literal backslash	\\	\

Requirement	Regex
Only digits	[0-9]+
Only alphabets	[a-zA-Z]+
Exactly 10 digits	[0-9]{10}
Lowercase letters only	[a-z]+
Uppercase letters only	[A-Z]+
Alphanumeric	[a-zA-Z0-9]+
Exactly 3 letters	[a-zA-Z]{3}
3 to 5 digits	[0-9]{3,5}
Optional s	s?
cat or dog	cat dog

Matcher Class

boolean matches()

Checks whether the entire string matches the pattern.

```
import java.util.regex.*;
```

```
class Demo1 {
```

```
    public static void main(String[] args) {
```

```
        Pattern p = Pattern.compile("[0-9]+");
```

```
        Matcher m = p.matcher("12345");
```

```
        System.out.println(m.matches());
```

```
    }
```

```
}
```

True

Because the complete string "12345" contains only digits.

```
Matcher m = p.matcher("123abc");
```

```
System.out.println(m.matches());
```

False

boolean find()

Searches for the next matching part of the string.

```
import java.util.regex.*;
```

```
class Demo2 {
```

```
    public static void main(String[] args) {
```

```
        Pattern p = Pattern.compile("[0-9]+");
```

```
        Matcher m = p.matcher("Java123");
```

```
        System.out.println(m.find());
```

```
    }
```

```
}
```

True

Because 123 is found inside "Java123"

```
Matcher m = p.matcher("Java");
```

```
System.out.println(m.find());
```

False

boolean find(int start)

Starts searching from the specified position.

<pre>import java.util.regex.*; class Demo3 { public static void main(String[] args) { Pattern p = Pattern.compile("[0-9]+"); Matcher m = p.matcher("Java123"); System.out.println(m.find(5)); } }</pre> J a v a 1 2 3 0 1 2 3 4 5 6 True	
<pre>Matcher m = p.matcher("Java123"); System.out.println(m.find(0));</pre> True	
<pre>Matcher m = p.matcher("JavaABC"); System.out.println(m.find(0));</pre> False	
String group() Returns the text that was matched. <pre>import java.util.regex.*; class Demo4 { public static void main(String[] args) { Pattern p = Pattern.compile("[0-9]+"); Matcher m = p.matcher("Java123ABC456"); while (m.find()) { System.out.println(m.group()); } } }</pre> 123 456	
int start() Returns the starting position of the current match. <pre>import java.util.regex.*;</pre>	

```

class Demo5 {
    public static void main(String[] args) {
        Pattern p = Pattern.compile("[0-9]+");
        Matcher m = p.matcher("Java123");
        m.find();
        System.out.println(m.start());
    }
}

```

J a v a 1 2 3

0 1 2 3 4 5 6

↑

Match

4

int end()

Returns the position immediately after the last character of the match.

```
import java.util.regex.*;
```

```

class Demo6 {
    public static void main(String[] args) {
        Pattern p = Pattern.compile("[0-9]+");
        Matcher m = p.matcher("Java123");
        m.find();
        System.out.println(m.end());
    }
}

```

J a v a 1 2 3

0 1 2 3 4 5 6

Match

The match ends after position 6, so end() returns 7.

7

int groupCount()

Returns the number of capturing groups in the regex.

```
import java.util.regex.*;
```

```

class Demo7 {
    public static void main(String[] args) {
        Pattern p = Pattern.compile("(Java) ([0-9]+)");
        Matcher m = p.matcher("Java123");
        System.out.println(m.groupCount());
    }
}

```

(Java) → Group 1

([0-9]+) → Group 2

2

Method	Purpose	Example output
matches()	Entire string matches?	true
find()	Find next match	true
find(5)	Find match starting from position 5	true
group()	Returns matched text	123
start()	Starting index of match	4
end()	Index immediately after match	7
groupCount()	Number of capturing groups	2

Matcher Method	Meaning	Return Value	Example
boolean matches()	Checks whether the entire input string matches the pattern	true / false	"123" with [0-9]+ → true
boolean find()	Searches for the next matching substring	true / false	"Java123" with [0-9]+ → true
boolean find(int start)	Searches for a matching substring starting from the specified index	true / false	"Java123", find(5) → true
String group()	Returns the currently matched substring	Matching String	"Java123" → "123"
int start()	Returns the starting index of the current match	Integer index	"Java123" → 4
int end()	Returns the index immediately after the current match	Integer index	"Java123" → 7
int groupCount()	Returns the number of capturing groups in the pattern	Integer	(Java)([0-9]+) → 2

```

import java.util.regex.*;

class MatcherDemo {
    public static void main(String[] args) {

```

```
Pattern p = Pattern.compile("(Java) ([0-9]+)");
Matcher m = p.matcher("Java 123 Java 456");
// 1. matches()
System.out.println("matches(): " + m.matches());
// 2. find()
System.out.println("find(): " + m.find());
// 3. group()
System.out.println("group(): " + m.group());
// 4. start()
System.out.println("start(): " + m.start());
// 5. end()
System.out.println("end(): " + m.end());
// 6. groupCount()
System.out.println("groupCount(): " + m.groupCount());
// 7. find(int start)
System.out.println("find(10): " + m.find(10));
}
}
```

matches(): false
find(): true
group(): Java 123
start(): 0
end(): 8
groupCount(): 2
find(10): true

Pattern Class

Pattern Method	Meaning	Return Type	Example
static Pattern compile(String regex)	Compiles the given regular expression into a Pattern object	Pattern	Pattern.compile("[0-9]+")
Matcher matcher(CharSequence input)	Creates a Matcher that applies this	Matcher	p.matcher("Java123")

	pattern to the given input		
static boolean matches(String regex, CharSequence input)	Checks whether the entire input matches the given regex	boolean	Pattern.matches("[0-9]+", "123") → true
String[] split(CharSequence input)	Splits the input string wherever the pattern matches	String[]	p.split("A,B,C")
String pattern()	Returns the regular expression used to create the Pattern	String	p.pattern() → [0-9]+

```

import java.util.regex.*;
class PatternDemo {
    public static void main(String[] args) {

        // 1. compile()
        Pattern p = Pattern.compile("[0-9]+");
        System.out.println("pattern(): " + p.pattern());
        //pattern() returns the regular expression stored inside a Pattern object.

        // 2. matcher()
        Matcher m = p.matcher("Java123");
        System.out.println("matcher() + find(): " + m.find());

        // 3. matches()
        System.out.println("matches(): "+ Pattern.matches("[0-9]+", "12345"));

        // 4. split()
        Pattern comma = Pattern.compile(",");
        String[] a = comma.split("Apple,Banana,Mango");

        System.out.println("split():");
        for (String s : a) {
            System.out.println(s);
        }
    }
}

```

```
    }  
  }  
}  
pattern(): [0-9]+  
matcher() + find(): true  
matches(): true  
split():  
Apple  
Banana  
Mango
```